

ORDERS TABLE _SQL AGGREGATE & CLAUSE OPERATIONS

M. Geethika (2520030485)

Question :

Perform SQL operations on an Orders table using WHERE, ORDER BY, GROUP BY, HAVING, SUM(), AVG(), MAX(), MIN(), COUNT(), and BETWEEN to analyse orders, customer purchases, salesman performance, and purchase amounts.

Step 1 : Create Database and Table

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Lists various schemas including aggregate_bank_db, bank_management_db, bookflow_db, joins_operations_db, physician_department_db, std_marks, sys, and triggers_db.
- SQL File Editor:** Contains the following SQL script:

```
-- Step 1: Create Database and Table
1 CREATE DATABASE IF NOT EXISTS orders_management_db;
2 USE orders_management_db;
3
4 DROP TABLE IF EXISTS orders;
5
6 CREATE TABLE orders (
7   ord_no INT PRIMARY KEY,
8   purch_amt DECIMAL(10,2),
9   ord_date DATE,
10  customer_id INT,
11  salesman_id INT
12 );
13
```
- Output Window:** Displays the execution results of the SQL script.

#	Time	Action	Message
1	10:23:21	CREATE DATABASE IF NOT EXISTS orders_management_db	1 row(s) affected
2	10:23:21	USE orders_management_db	0 row(s) affected
3	10:23:21	DROP TABLE IF EXISTS orders	0 row(s) affected, 1 warning(s): 1051 Unknown table 'orders_management_db.orders'
4	10:23:21	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ...	0 row(s) affected

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2' x

Limit to 1000 rows

```

13
14 -- Step 2: Insert Data --
15 INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) VALUES
16 (70001, 150.50, '2012-10-05', 3005, 5002),
17 (70009, 270.65, '2012-09-10', 3001, 5005),
18 (70002, 65.26, '2012-10-05', 3002, 5001),
19 (70004, 110.50, '2012-08-17', 3009, 5003),
20 (70007, 948.50, '2012-09-10', 3005, 5002),
21 (70005, 2400.60, '2012-07-27', 3007, 5001),
22 (70006, 5760.00, '2012-09-10', 3002, 5001),
23 (70010, 1983.43, '2012-10-10', 3004, 5006),
24 (70003, 2480.40, '2012-10-10', 3009, 5003),
25 (70012, 250.45, '2012-06-27', 3008, 5002),
26 (70011, 75.29, '2012-08-17', 3003, 5007),
27 (70013, 3045.60, '2012-04-25', 3002, 5001);
28
29

```

Output

Action Output

#	Time	Action	Message
4	10:23:21	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2)...	0 row(s) affected
5	10:29:48	CREATE DATABASE IF NOT EXISTS orders_management_db	1 row(s) affected, 1 warning(s): 1007 Can't create dat
6	10:29:48	USE orders_management_db	0 row(s) affected
7	10:29:48	DROP TABLE IF EXISTS orders	0 row(s) affected
8	10:29:48	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2)...	0 row(s) affected
9	10:29:48	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) VA...	12 row(s) affected Records: 12 Duplicates: 0 Warni

Step 2 : WHERE

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2' x

Limit to 1000 rows

```

25 (70012, 250.45, '2012-06-27', 3008, 5002),
26 (70011, 75.29, '2012-08-17', 3003, 5007),
27 (70013, 3045.60, '2012-04-25', 3002, 5001);
28
29 -- Step 3: WHERE Clause --
30 -- Purchase amount greater than 2000
31 SELECT *
32 FROM orders
33 WHERE purch_amt > 2000;
34

```

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

orders 1 x

Output

Action Output

#	Time	Action	Message
10	10:30:49	CREATE DATABASE IF NOT EXISTS orders_management_db	1 row(s) affected, 1 warning(s): 1007 Can't create database 'orders
11	10:30:49	USE orders_management_db	0 row(s) affected
12	10:30:49	DROP TABLE IF EXISTS orders	0 row(s) affected
13	10:30:49	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2)...	0 row(s) affected
14	10:30:49	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
15	10:30:49	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned

SQL File 2*

```

29 -- Step 3: WHERE Clause --
30 -- Purchase amount greater than 2000
31 SELECT *
32 FROM orders
33 WHERE purch_amt > 2000;
34
35 -- Step 4: WHERE Clause with Date --
36 SELECT *
37 FROM orders
38 WHERE ord_date = '2012-09-10';

```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005

orders 2 orders 3 x

Output

Action Output

#	Time	Action	Message
17	10:31:24	USE orders_management_db	0 row(s) affected
18	10:31:24	DROP TABLE IF EXISTS orders	0 row(s) affected
19	10:31:24	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2...	0 row(s) affected
20	10:31:24	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
21	10:31:24	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
22	10:31:24	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned

SQL File 2*

```

39
40 -- Step 5: WHERE Clause with Salesman --
41 SELECT *
42 FROM orders
43 WHERE salesman_id = 5001;

```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

orders 4 orders 5 orders 6 x

Output

Action Output

#	Time	Action	Message
25	10:32:02	DROP TABLE IF EXISTS orders	0 row(s) affected
26	10:32:02	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2...	0 row(s) affected
27	10:32:02	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
28	10:32:02	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
29	10:32:02	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
30	10:32:02	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned

Step 3 : ORDER BY

MySQL Workbench

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

44
45 -- Step 6: ORDER BY Purchase Amount
46 -- Highest to lowest --
47 • SELECT *
48 FROM orders
49 ORDER BY purch_amt DESC;

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007

orders 7 orders 8 orders 9 orders 10 x

Apply Revert Context Help

Output

Action Output

#	Time	Action	Message
34	10:33:40	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2...	0 row(s) affected
35	10:33:40	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
36	10:33:40	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
37	10:33:40	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
38	10:33:40	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
39	10:33:40	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned

Object Info Session

MySQL Workbench

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

50
51 -- Step 7: ORDER BY Order Date
52 -- Oldest to newest --
53 • SELECT *
54 FROM orders
55 ORDER BY ord_date ASC;

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003

orders 11 x

Apply Revert Context Help

Output

Action Output

#	Time	Action	Message
35	10:33:40	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
36	10:33:40	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
37	10:33:40	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
38	10:33:40	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
39	10:33:40	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
40	10:34:53	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned

Object Info Session

Step 4 : AGGREGATE FUNCTIONS

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

55 ORDER BY ord_date ASC;

56

57 -- Step 8: SUM()

58 -- Total revenue --

59 • SELECT SUM(purch_amt) AS total_revenue

60 FROM orders;

61

Result Grid

total_revenue
17541.18

Administration Schemas

Information

No object selected

orders 12 orders 13 orders 14 orders 15 orders 16 Result 17 x

Output

Action Output

#	Time	Action	Message
✓ 46	10:36:08	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
✓ 47	10:36:08	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
✓ 48	10:36:08	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 49	10:36:08	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 50	10:36:08	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 51	10:36:08	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

59 • SELECT SUM(purch_amt) AS total_revenue

60 FROM orders;

61

62 -- Step 9: AVG()

63 -- Average order amount --

64 • SELECT AVG(purch_amt) AS average_order

65 FROM orders;

Result Grid

average_order
1461.765000

Administration Schemas

Information

No object selected

orders 18 orders 19 orders 20 orders 21 orders 22 Result 23 Result 24 x

Output

Action Output

#	Time	Action	Message
✓ 58	10:36:51	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
✓ 59	10:36:51	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 60	10:36:51	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 61	10:36:51	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 62	10:36:51	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 63	10:36:51	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

SQL File 2* x

```

64 • SELECT AVG(purch_amt) AS average_order
65 FROM orders;
66
67 -- Step 10: MAX()
68 -- Highest order amount --
69 • SELECT MAX(purch_amt) AS highest_order
70 FROM orders;

```

Result Grid

highest_order
5760.00

Export: Wrap Cell Content: [F1](#)

orders 25 orders 26 orders 27 orders 28 orders 29 Result 30 Result 31 Result 32 x

Output

Action Output

#	Time	Action	Message
71	10:37:26	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
72	10:37:26	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
73	10:37:26	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
74	10:37:26	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
75	10:37:26	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
76	10:37:26	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

SQL File 2* x

```

69 • SELECT MAX(purch_amt) AS highest_order
70 FROM orders;
71
72 -- Step 11: MIN()
73 -- Lowest order amount --
74 • SELECT MIN(purch_amt) AS lowest_order
75 FROM orders;

```

Result Grid

lowest_order
65.26

Export: Wrap Cell Content: [F1](#)

orders 33 orders 34 orders 35 orders 36 orders 37 Result 38 Result 39 Result 40 Result 41

Output

Action Output

#	Time	Action	Message
85	10:38:09	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
86	10:38:09	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
87	10:38:09	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
88	10:38:09	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
89	10:38:09	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
90	10:38:09	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

```

74 • SELECT MIN(purch_amt) AS lowest_order
75 FROM orders;
76
77 -- Step 12: COUNT()
78 -- Total number of orders
79 • SELECT COUNT(*) AS total_orders
80 FROM orders;

```

Result Grid Filter Rows: Export: Wrap Cell Content:

	total_orders
▶	12

Administration Schemas

Information

No object selected

Object Info Session

ders 42 orders 43 orders 44 orders 45 orders 46 Result 47 Result 48 Result 49 Result 50

Output

Action Output

#	Time	Action	Message
✓ 101	10:39:10	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 102	10:39:10	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 103	10:39:10	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 104	10:39:10	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 105	10:39:10	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 106	10:39:10	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned

Step 5 : GROUP BY

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

Limit to 1000 rows

```

80 FROM orders;
81
82 -- Step 13: GROUP BY Salesman
83 -- Total sales made by each salesman
84 • SELECT salesman_id, SUM(purch_amt) AS total_sales
85 FROM orders
86 GROUP BY salesman_id;

```

Result Grid Filter Rows: Export: Wrap Cell Content:

	salesman_id	total_sales
▶	5002	1349.45
	5001	11271.46
	5003	2590.90
	5005	270.65
	5006	1983.43
	5007	75.29

Administration Schemas

Information

No object selected

Object Info Session

orders 54 orders 55 orders 56 Result 57 Result 58 Result 59 Result 60 Result 61 Result

Output

Action Output

#	Time	Action	Message
✓ 117	10:40:10	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 118	10:40:10	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 119	10:40:10	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 120	10:40:10	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 121	10:40:10	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 122	10:40:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

```

86      GROUP BY salesman_id;
87
88      -- Step 14: GROUP BY Customer
89      -- Total purchase made by each customer
90      SELECT customer_id, SUM(purch_amt) AS total_purchase
91      FROM orders
92      GROUP BY customer_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

orders 66 orders 67 Result 68 Result 69 Result 70 Result 71 Result 72 Result 73 Result 74

Output

Action Output

#	Time	Action	Message
✓ 134	10:41:01	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 135	10:41:01	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 136	10:41:01	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 137	10:41:01	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 138	10:41:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY...	6 row(s) returned
✓ 139	10:41:01	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY...	8 row(s) returned

Object Info Session

MySQL workbench

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

```

92      GROUP BY customer_id;
93
94      -- Step 15: GROUP BY + MAX()
95      -- Highest purchase made by each customer
96      SELECT customer_id, MAX(purch_amt) AS highest_purchase
97      FROM orders
98      GROUP BY customer_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	highest_purchase
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Result 81 Result 82 Result 83 Result 84 Result 85 Result 86 Result 87 Result 88

Output

Action Output

#	Time	Action	Message
✓ 154	10:42:29	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 155	10:42:29	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 156	10:42:29	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY...	6 row(s) returned
✓ 157	10:42:29	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY...	8 row(s) returned
✓ 158	10:42:29	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY...	8 row(s) returned
✓ 159	10:42:29	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY...	1 row(s) returned

Object Info Session

Step 6 : GROUP BY +HAVING

The screenshot shows a SQL IDE interface with a menu bar (File, Edit, View, Query, Database, Server, Tools, Scripting, Help) and a toolbar. The left sidebar contains a 'SCHEMAS' tree with a search bar and a list of databases: aggregate_bank_db, bank_management_db, bookflow_db, joins_operations_db, physician_department_db, std_marks, sys, and triggers_db. The main editor displays a SQL script in 'SQL File 2*' with the following content:

```
98 GROUP BY customer_id;
99
100 -- Step 16: GROUP BY + HAVING
101 -- Salesmen whose total sales are greater than 3000
102 • SELECT salesman_id, SUM(purch_amt) AS total_sales
103 FROM orders
104 GROUP BY salesman_id HAVING SUM(purch_amt) > 3000;
```

Below the script, the 'Result Grid' shows the output of the query:

salesman_id	total_sales
5001	11271.46

The bottom section of the IDE shows a log of executed queries. The 'Output' tab is active, displaying a table with columns: #, Time, Action, and Message. The log contains several entries, with the last one being the query shown in the editor:

#	Time	Action	Message
154	10:42:29	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
155	10:42:29	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
156	10:42:29	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY...	6 row(s) returned
157	10:42:29	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROU...	8 row(s) returned
158	10:42:29	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GR...	8 row(s) returned
159	10:42:29	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY...	1 row(s) returned

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2*

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

Object Info Session

SQL File 2*

Limit to 1000 rows

```

104 GROUP BY salesman_id HAVING SUM(purch_amt) > 3000;
105
106 -- Step 17: GROUP BY + HAVING
107 -- Customers whose total purchase is greater than 2500
108 SELECT customer_id, SUM(purch_amt) AS total_purchase
109 FROM orders
110 GROUP BY customer_id HAVING SUM(purch_amt) > 2500;
111
112 -- Step 18: GROUP BY + HAVING
113 -- Customers having more than one order
114 SELECT customer_id, COUNT(*) AS total_orders

```

Result Grid

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1082.42

Result 97 Result 98 Result 99 Result 100 Result 101 x Result 102 Result 103 Result 104 Result 105

Output

Action Output

#	Time	Action	Message
✓ 167	10:44:12	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 168	10:44:12	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 169	10:44:12	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 170	10:44:12	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 171	10:44:12	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 172	10:44:12	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 173	10:44:12	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 2* x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys
- triggers_db

Administration Schemas

Information

No object selected

Object Info Session

104 GROUP BY salesman_id HAVING SUM(purch_amt) > 3000;

105

106 -- Step 17: GROUP BY + HAVING

107 -- Customers whose total purchase is greater than 2500

108 • SELECT customer_id, SUM(purch_amt) AS total_purchase

109 FROM orders

110 GROUP BY customer_id HAVING SUM(purch_amt) > 2500;

111

112 -- Step 18: GROUP BY + HAVING

113 -- Customers having more than one order

114 • SELECT customer_id, COUNT(*) AS total_orders

Result Grid

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1002.45

Result 97 Result 98 Result 99 Result 100 Result 101 x Result 102 Result 103 Result 104 Result 105

Output

Action Output

#	Time	Action	Message
✓ 167	10:44:12	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 168	10:44:12	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 169	10:44:12	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 170	10:44:12	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 171	10:44:12	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 172	10:44:12	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 173	10:44:12	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned

Step 7: GROUP BY +HAVING+ORDER BY

SQL File 2*

```

117
118 -- Step 19: GROUP BY + HAVING + ORDER BY
119 -- Customers with total purchase greater than 1000
120 • SELECT customer_id, SUM(purch_amt) AS total_purchase
121 FROM orders
122 GROUP BY customer_id HAVING SUM(purch_amt) > 1000
123 ORDER BY total_purchase DESC;
124
125 -- Step 20: BETWEEN + HAVING
126 -- Customers whose maximum purchase is between 2000 and 6000
127 • SELECT customer_id, MAX(purch_amt) AS max_purchase
128 FROM orders GROUP BY customer_id HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
129

```

Result Grid

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65

Output

#	Time	Action	Message
180	10:44:12	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id	3 row(s) returned
181	10:47:09	CREATE DATABASE IF NOT EXISTS orders_management_db	1 row(s) affected, 1 warning(s): 1007 Can't create database
182	10:47:09	USE orders_management_db	0 row(s) affected
183	10:47:09	DROP TABLE IF EXISTS orders	0 row(s) affected
184	10:47:09	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),	0 row(s) affected
185	10:47:09	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
186	10:47:09	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned

Step 8 : BETWEEN + HAVING

SQL File 2*

```

123 ORDER BY total_purchase DESC;
124
125 -- Step 20: BETWEEN + HAVING
126 -- Customers whose maximum purchase is between 2000 and 6000
127 • SELECT customer_id, MAX(purch_amt) AS max_purchase
128 FROM orders GROUP BY customer_id HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
129

```

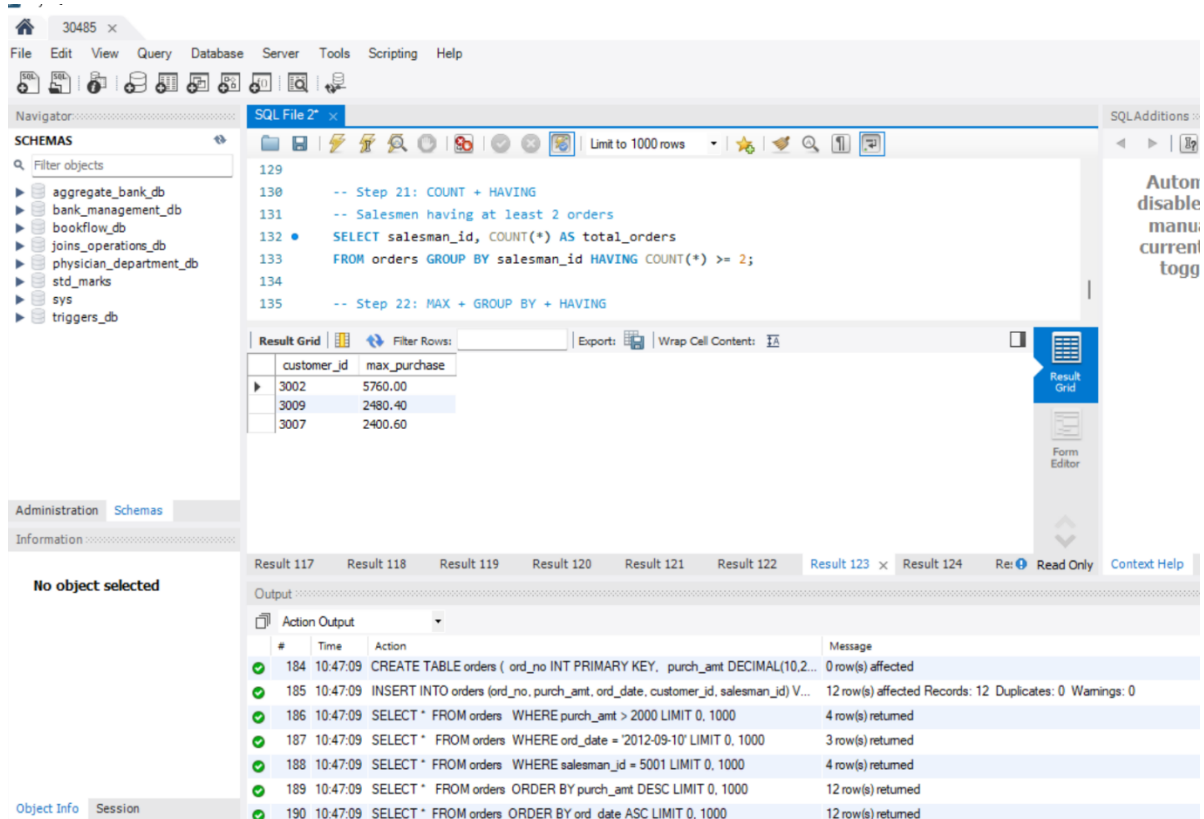
Result Grid

customer_id	total_purchase
3002	8870.86
3009	2590.90

Output

#	Time	Action	Message
184	10:47:09	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),	0 row(s) affected
185	10:47:09	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
186	10:47:09	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
187	10:47:09	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
188	10:47:09	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
189	10:47:09	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
190	10:47:09	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned

Step 9 : COUNT+HAVING



SQL File 2*

```
129
130 -- Step 21: COUNT + HAVING
131 -- Salesmen having at least 2 orders
132 * SELECT salesman_id, COUNT(*) AS total_orders
133 FROM orders GROUP BY salesman_id HAVING COUNT(*) >= 2;
134
135 -- Step 22: MAX + GROUP BY + HAVING
```

Result Grid

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

Output

#	Time	Action	Message
184	10:47:09	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2...	0 row(s) affected
185	10:47:09	INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) V...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
186	10:47:09	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
187	10:47:09	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
188	10:47:09	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
189	10:47:09	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
190	10:47:09	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned

Step 10 : MAX+GROUP BY + HAVING

The screenshot shows the SQL Developer interface with a SQL query and its results. The query is as follows:

```

132 • SELECT salesman_id, COUNT(*) AS total_orders
133 FROM orders GROUP BY salesman_id HAVING COUNT(*) >= 2;
134
135 -- Step 22: MAX + GROUP BY + HAVING
136 -- Dates where highest purchase is greater than 2000
137 • SELECT ord_date, MAX(purch_amt) AS highest_purchase
138 FROM orders GROUP BY ord_date HAVING MAX(purch_amt) > 2000;

```

The results are displayed in a table with two columns: `ord_date` and `highest_purchase`.

ord_date	highest_purchase
2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60

The interface also shows a 'Schemas' pane on the left with a list of databases: `aggregate_bank_db`, `bank_management_db`, `bookflow_db`, `joins_operations_db`, `physician_department_db`, `std_marks`, `sys`, and `triggers_db`. The 'Administration' tab is selected, and the 'Schemas' sub-tab is active. The 'Information' pane at the bottom shows 'No object selected'.

